

Gradient Boosting for Regression: the Maths

Contents

1. Overview	2
2. A model is a function	3
3. Additive modelling: a complex function as a sum of simple ones	3
3.1 When lines and polynomials are not enough	3
3.2 Break the function into parts	3
3.3 Boosting is stage-wise additive modelling	4
4. The ingredients: training data and a differentiable loss	4
4.1 Three startups	5
4.2 The loss function	5
5. Step 1: the best constant is the mean	5
6. Step 2(a): pseudo-residuals are negative gradients	6
7. Step 2(b): fit a tree; it creates terminal regions	7
8. Step 2(c): the best value for each leaf	8
9. Step 2(d): update the model	9
10. Step 3: the output	9
11. Summary	11
12. Key terms	11

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Gradient descent ([Note 57](#)); Log loss (binary cross entropy) ([Note 73](#)); Sigmoid function ([Note 74](#)); Regression trees ([Note 99](#)); Boosting ([Note 119](#)); Learning rate ([Note 120](#)).
- **Leads to:** XGBoost ([Note 123](#)).
- **Compare with:** AdaBoost ([Note 118](#)).

1. Overview

Key point: Gradient boosting is one algorithm with three steps: start from the best constant, then repeatedly compute pseudo-residuals, fit a tree to them and give each leaf its best value, and finally output the sum.

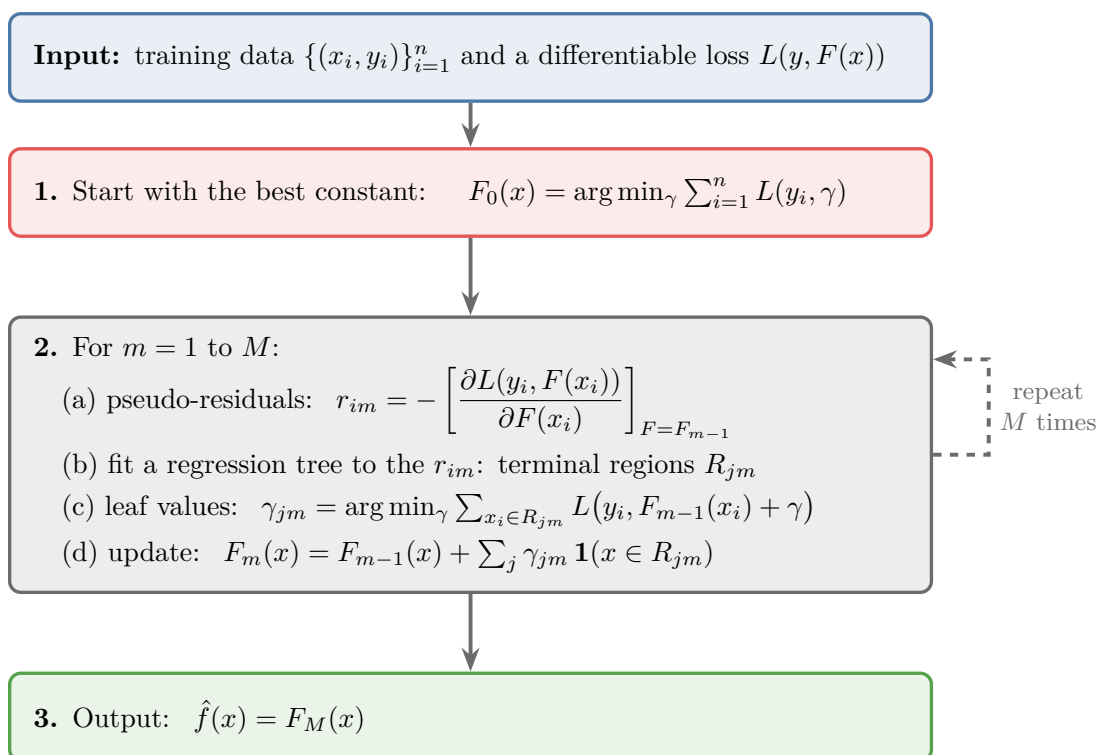


Figure 1: The gradient boosting algorithm (after The Elements of Statistical Learning, Algorithm 10.3)

The [gradient boosting intuition Note](#) ran gradient boosting by hand: the mean, residuals, trees on the residuals, a learning rate. Figure 1 is the same procedure written as the formal

algorithm from the textbook *The Elements of Statistical Learning*. This Note explains every line of it and runs each step on three startups.

The same algorithm also solves classification; only the loss function changes ([gradient boosting classification Note](#)). Before the steps, we need one idea that all boosting algorithms share: additive modelling.

2. A model is a function

Key point: Every ML model is a function $y = f(x)$ that maps the input columns to the output column.

Whatever the algorithm, an ML model learns a mathematical relationship between the inputs and the output. With one input x (say CGPA) and an output y (say package), the model is a function $y = f(x)$. With several inputs, it is $y = f(x_1, x_2, x_3)$.

Linear regression learns a straight line ([simple linear regression Note](#)): a very simple function. A decision tree is also a function, made of if-else rules ([regression trees Note](#)). Learning means finding the function that matches the data.

3. Additive modelling: a complex function as a sum of simple ones

Key point: When the true function is too complex for a line or a polynomial, we build it as a sum of simple functions, added one at a time. Boosting does exactly this, with trees as the simple functions.

3.1 When lines and polynomials are not enough

Key point: A line misses curved data; high-degree polynomials swing wildly at the edges.

Some data follows a pattern that wiggles as it rises. A straight line cannot follow it. Polynomial regression ([polynomial regression Note](#)) can bend, but a high degree makes the curve shoot up or down at the edges of the data, which lowers the test R^2 .

Extra: This edge problem has a name, **Runge's phenomenon**: when a polynomial of high degree is fitted through equally spaced points, it oscillates strongly near the ends of the interval.

3.2 Break the function into parts

Key point: $y = x + \sin x$ is hard to guess as a whole, but easy as the sum of a line and a wave.

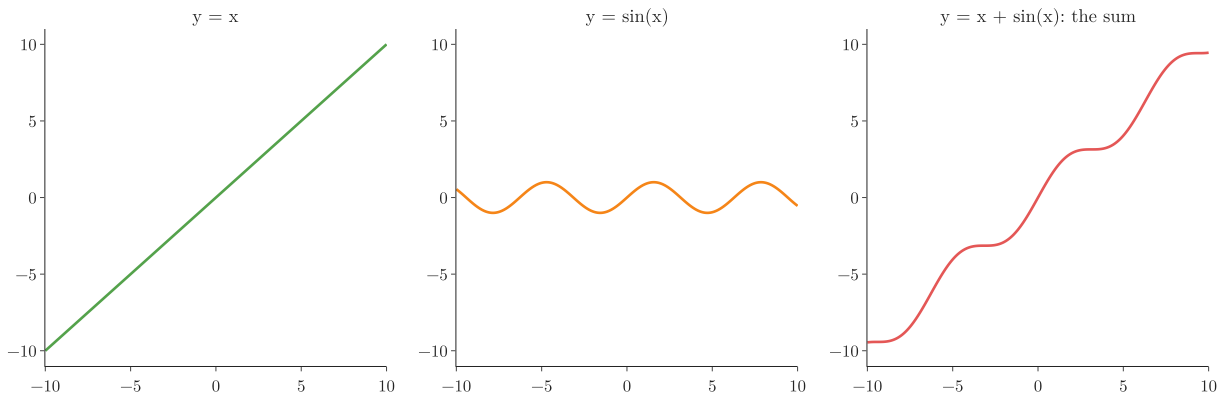


Figure 2: A complex function as the sum of two simple ones: a straight line plus a sine wave

Figure 2 shows such a function: it rises like a line and wiggles like a wave. It is in fact the sum of two simple functions, $y = x$ and $y = \sin x$. If we did not know this, we could still approach it step by step: first catch the rising trend, then catch the wiggle that is left over, and add the two.

This is **additive modelling**: we approximate a complex function by adding simple functions, each one capturing part of what the others missed.

$$F(x) = f_0(x) + f_1(x) + f_2(x) + \cdots + f_M(x)$$

3.3 Boosting is stage-wise additive modelling

Key point: In boosting, each added function is a model (a tree), and they are added in stages, one after another.

All boosting algorithms build this kind of sum. The [AdaBoost intuition Note](#), section 3, called it a stage-wise additive model: “additive” because the final model is a sum, “stage-wise” because the parts are added one per stage.

In gradient boosting, f_0 is a constant and every later f_m is a regression tree, which is also a function. Each new tree moves the sum closer to the true function, which is why the training error falls with every stage.

4. The ingredients: training data and a differentiable loss

Key point: The algorithm needs two inputs: the training rows (x_i, y_i) , and a loss function $L(y, F(x))$ that we can differentiate.

4.1 Three startups

Key point: Three rows, three input columns (spending), one output (profit): $n = 3$.

We use three rows of the 50 Startups data, a regression dataset of US companies (amounts in thousands):

Startup	R&D spend	Administration	Marketing	Profit
1	165.35	136.90	471.78	192.26
2	100.67	91.79	249.74	144.26
3	28.66	127.06	201.13	90.71

The notation $\{(x_i, y_i)\}_{i=1}^n$ in Figure 1 simply means this table. Here $n = 3$; x_1 is the first startup's three spends together, and $y_1 = 192.26$ is its profit.

4.2 The loss function

Key point: We use half the squared error, $L = \frac{1}{2}(y - F(x))^2$. Any loss works, as long as it can be differentiated.

A **loss function** measures how wrong a prediction is ([regression metrics Note](#)). In Figure 1 it is written $L(y, F(x))$, where $F(x)$ is the model's output, $\hat{y}$. Gradient boosting accepts any loss we can differentiate at every point; this flexibility is one of its strengths.

For regression we take the squared error with a factor of one half.

1. **In words:** half of the squared difference between the true value and the prediction.
2. **Formula:**

$$L(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2$$

3. **Example:** startup 1 with a prediction of 142.41:

$$L = \frac{1}{2}(192.26 - 142.41)^2 = \frac{1}{2} \times 2485.02 = 1242.51$$

The $\frac{1}{2}$ is for convenience: when we differentiate, the 2 from the square cancels it. It does not change which model is best. If line 1 has squared error 10 and line 2 has 20, halving gives 5 and 10, and line 1 is still the better one.

5. Step 1: the best constant is the mean

Key point: F_0 is the single number γ that makes the total loss smallest. For half the squared error, setting the derivative to zero gives the mean of y .

Step 1 finds the first function, F_0 . It does not depend on x : it is one number for every row, a leaf on its own.

1. **In words:** choose the constant γ (gamma) that makes the sum of the losses over all rows as small as possible. “arg min $_{\gamma}$ ” means “the value of γ that minimises”.
2. **Formula:**

$$F_0 = \arg \min_{\gamma} \sum_{i=1}^n \frac{1}{2} (y_i - \gamma)^2$$

3. **Example:** for our three startups we need the γ that minimises $\frac{1}{2}[(192.26 - \gamma)^2 + (144.26 - \gamma)^2 + (90.71 - \gamma)^2]$. The derivation below gives $\gamma = 142.41$.

To find the minimum, we differentiate with respect to γ and set the result to zero, as for linear regression ([linear regression maths Note](#)). By the chain rule, the 2 from the square cancels the $\frac{1}{2}$, and the inner derivative of $(y_i - \gamma)$ is -1 :

$$\frac{d}{d\gamma} \sum_{i=1}^n \frac{1}{2} (y_i - \gamma)^2 = - \sum_{i=1}^n (y_i - \gamma) = 0$$

Multiplying by -1 and writing out the sum for $n = 3$:

$$(\gamma - 192.26) + (\gamma - 144.26) + (\gamma - 90.71) = 0 \quad \Rightarrow \quad 3\gamma = 427.23 \quad \Rightarrow \quad \gamma = 142.41$$

That is the mean of the profits. So, with the squared-error loss, the first model is always the mean of the output column, as the intuition Note claimed. With another loss, step 1 gives another constant.

6. Step 2(a): pseudo-residuals are negative gradients

Key point: For every row, the pseudo-residual is minus the derivative of the loss with respect to the current prediction. For half the squared error, that is exactly actual minus predicted.

Step 2 is a loop that runs M times, once per tree. With $M = 50$ we get 50 trees, f_1 to f_{50} . Inside the loop, the index m is the tree we are building and i is the row.

1. **In words:** differentiate the loss with respect to the prediction $F(x_i)$, plug in the current model F_{m-1} , and flip the sign.
2. **Formula:**

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

For $L = \frac{1}{2}(y_i - F(x_i))^2$, the chain rule gives $\frac{\partial L}{\partial F(x_i)} = -(y_i - F(x_i))$, so

$$r_{im} = y_i - F_{m-1}(x_i)$$

3. **Example:** for the first tree ($m = 1$) the current model is $F_0 = 142.41$:

$$r_{11} = 192.26 - 142.41 = 49.85, \quad r_{21} = 144.26 - 142.41 = 1.85, \quad r_{31} = 90.71 - 142.41 = -51.70$$

The derivative of the loss is its **gradient**, the same quantity gradient descent follows ([gradient descent Note](#)). Minus the gradient points in the direction that lowers the loss fastest. For the squared error it happens to equal the ordinary residual, actual minus predicted; for other losses it does not, which is why it is called a *pseudo*-residual.

Extra: This is where “gradient” in gradient boosting comes from. Gradient descent changes a parameter by a small step against the gradient. Gradient boosting changes the *predictions* by a small step against the gradient: each tree learns the negative gradient, and adding it (times the learning rate) is one step of gradient descent on the predictions. It is gradient descent in function space.

7. Step 2(b): fit a tree; it creates terminal regions

Key point: A regression tree is trained with the inputs as x and the pseudo-residuals as the target. Its leaves cut the input space into terminal regions R_{jm} .

Next we train a regression tree: the three spends are the inputs, the column r_{i1} is the target. With only three rows we allow a single split (`max_depth=1`); real trees usually have 8 to 32 leaves.

The tree splits on R&D spend at 64.67 (halfway between 28.66 and 100.67). Startup 3 goes to one leaf; startups 1 and 2 go to the other.

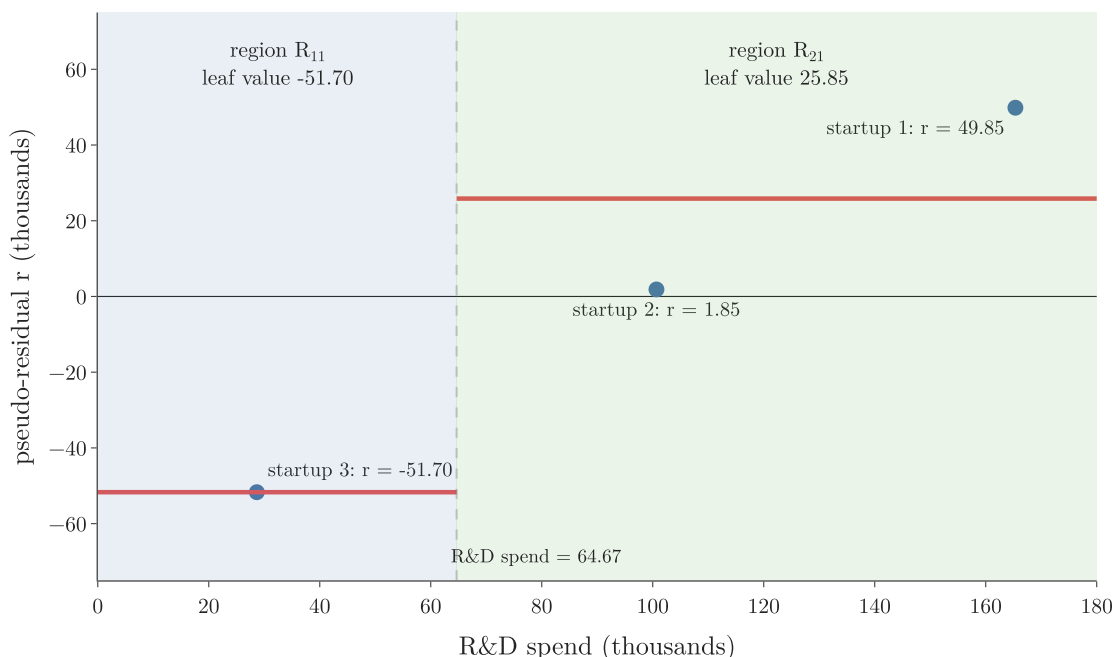


Figure 3: The first tree on the pseudo-residuals: one split on R&D spend makes two terminal regions, each with its leaf value

Each leaf covers a part of the input space, called a **terminal region** R_{jm} : region j of tree m .

In Figure 3, R_{11} is “R&D spend ≤ 64.67 ” and R_{21} is “R&D spend > 64.67 ”. A deeper tree would cut more regions, up to J_m for tree m .

The tree’s own prediction in each region is the mean of the targets there: -51.70 on the left, $(49.85 + 1.85)/2 = 25.85$ on the right.

8. Step 2(c): the best value for each leaf

Key point: Each leaf gets the constant γ_{jm} that best corrects the current model for the rows in that leaf. For the squared error, this is the mean of the residuals in the leaf, the same value the tree already gives.

Step 2(c) does not simply trust the tree’s leaf values. For each region it solves a small version of step 1, using only the rows that fall in that region.

1. **In words:** for each leaf, find the amount γ which, added to the current predictions of the rows in that leaf, makes their loss smallest.

2. **Formula:**

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$

3. **Example:** region R_{11} holds only startup 3. Setting the derivative of $\frac{1}{2}(90.71 - 142.41 - \gamma)^2$ to zero:

$$-(90.71 - 142.41 - \gamma) = 0 \quad \Rightarrow \quad \gamma_{11} = 90.71 - 142.41 = -51.70$$

Region R_{21} holds startups 1 and 2, so the sum has two terms:

$$-[(192.26 - 142.41 - \gamma) + (144.26 - 142.41 - \gamma)] = 0 \quad \Rightarrow \quad 51.70 - 2\gamma = 0 \quad \Rightarrow \quad \gamma_{21} = 25.85$$

Both values equal the tree’s own leaf values. That is no coincidence: with the squared error, the minimising γ is always the mean of the residuals in the leaf, which is exactly what a regression tree predicts. With another loss, step 2(c) gives different values, and then the step matters.

Extra: With the absolute error $L = |y - F(x)|$ (scikit-learn’s `loss="absolute_error"`), step 1 gives the **median**, 144.26. The pseudo-residuals become the signs of $y - F$: $+1$, 0 and -1 , so the tree learns only the direction of each mistake. Step 2(c) then sets each leaf to the median of the actual residuals in it: -53.55 for startup 3, and for startups 1 and 2 (residuals 48 and 0) any value between 0 and 48 is a median; scikit-learn takes 0. The tree’s own leaf values (the means of the signs, -1 and 0.5) are replaced. The Notebook reproduces these numbers.

9. Step 2(d): update the model

Key point: Add the leaf value of the region each row falls in to the current model: $F_m = F_{m-1} + \gamma_{jm}$ (in practice times the learning rate).

1. **In words:** each row's new prediction is its old prediction plus the value of the leaf it lands in.
2. **Formula:**

$$F_m(x) = F_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} \mathbf{1}(x \in R_{jm})$$

$\mathbf{1}(x \in R_{jm})$ is 1 if x lies in region R_{jm} and 0 otherwise, so the sum just picks the value of the one leaf x falls in.

3. **Example:** startups 1 and 2 are in R_{21} , startup 3 in R_{11} :

$$F_1 = 142.41 + 25.85 = 168.26 \quad (\text{startups 1, 2}), \quad F_1 = 142.41 - 51.70 = 90.71 \quad (\text{startup 3})$$

In practice each tree is multiplied by the learning rate η before it is added, as in the [gradient boosting intuition Note](#), section 8: $F_m(x) = F_{m-1}(x) + \eta \sum_j \gamma_{jm} \mathbf{1}(x \in R_{jm})$. With $\eta = 0.1$ the predictions become 144.995, 144.995 and 137.24. scikit-learn's `GradientBoostingRegressor` with one tree of depth 1 gives exactly these numbers for both learning rates (Notebook).

Then the loop goes back to step 2(a) with $m = 2$: new pseudo-residuals from F_1 , a new tree, new leaf values, a new update.

10. Step 3: the output

Key point: After M rounds, the model is the constant plus all the trees: $F_M(x) = F_0 + f_1(x) + \dots + f_M(x)$.

Unrolling the updates shows the additive model of section 3:

$$F_M(x) = F_0 + f_1(x) + f_2(x) + \dots + f_M(x)$$

where each $f_m(x)$ is tree m with its leaf values γ_{jm} (times η). To predict for a new startup, we send it down every tree, collect one leaf value from each, and add them to F_0 .

Python: The loss is a parameter of `GradientBoostingRegressor`.

```
from sklearn.ensemble import GradientBoostingRegressor
```

```
gbr = GradientBoostingRegressor(  
    loss="squared_error",      # or "absolute_error", "huber", "quantile"  
    n_estimators=100,          # M, the number of trees  
    learning_rate=0.1,         # eta  
    max_depth=3)               # size of each tree  
gbr.fit(X, y)
```

"huber" is squared error for small residuals and absolute error for large ones, so outliers

pull less; `"quantile"` predicts a chosen quantile instead of the mean. Old names such as `"ls"` and `"lad"` were removed from scikit-learn and now raise an error.

11. Summary

Step	What it does	Squared error gives	Startups
1	best constant F_0	the mean of y	142.41
2(a)	pseudo-residuals r_{im} = minus the gradient	$y_i - F_{m-1}(x_i)$	49.85, 1.85, -51.70
2(b)	regression tree on r_{im}	terminal regions R_{jm}	R&D ≤ 64.67 or not
2(c)	best value γ_{jm} per leaf	mean residual of the leaf	-51.70, 25.85
2(d)	$F_m = F_{m-1} + \eta \cdot$ leaf value		168.26, 168.26, 90.71 ($\eta = 1$)
3	output F_M	$F_0 +$ all trees	

- An ML model is a function; additive modelling builds a complex function as a sum of simple ones, and boosting adds them in stages.
- The algorithm needs training data and a differentiable loss; half the squared error is the standard choice for regression.
- Step 1 and step 2(c) are both “find the best constant”: over all rows for F_0 , over one leaf’s rows for γ_{jm} .
- The pseudo-residual is the negative gradient of the loss; for squared error it is the ordinary residual, which is why the intuition Note could use actual minus predicted.
- With squared error, the leaf values equal the tree’s own means; with other losses (absolute, Huber, log loss) step 2(c) changes them.

12. Key terms

Term	Meaning
Additive modelling	Building a complex function as a sum of simple functions, each capturing part of what the others missed
Runge’s phenomenon	The large swings of a high-degree polynomial near the ends of the interval it is fitted on
Differentiable loss	A loss function whose derivative exists at every point, so it can be minimised with derivatives
Arg min	The value of a variable that makes an expression smallest, written $\arg \min$
Negative gradient	Minus the derivative of the loss with respect to the prediction; the direction that lowers the loss fastest
Terminal region	The part of the input space that ends in one leaf of a tree, written R_{jm} for leaf j of tree m

Term	Meaning
Leaf value (γ_{jm})	The constant a leaf adds to the model, chosen to minimise the loss of the rows in that leaf